

Параллельное программирование

Отчет

Лабораторная работа №6: Параллельная реализация решения уравнения теплопроводности

Выполнил: группа 24940, МIRONЕНКО Павел

Дата: 04.06.2026

Цель работы

Разработать и реализовать итерационный алгоритм решения стационарного уравнения теплопроводности (метод Якоби, пятиточечный разностный шаблон) для двумерной области. Расчеты проводятся на равномерных сетках размерами 128×128 , 256×256 , 512×512 и 1024×1024 . Граничные условия формируются посредством линейной интерполяции между угловыми точками, имеющими значения 10, 20, 30 и 20. Внутренние узлы сетки инициализируются нулями.

Необходимо достичь точности вычислений

10^{-6} при лимите в 10^6 итераций. Также требуется портировать код на графический ускоритель с применением директив OpenACC, провести оптимизацию обменов памятью и сохранить итоговую матрицу температур для верификации корректности работы алгоритма.

Используемый компилятор

Для компиляции кода использовался компилятор `nvc++` (входит в состав NVIDIA HPC SDK). Сборка осуществлялась с флагами `-acc` (для включения поддержки OpenACC) и `-Minfo=all` (для получения подробного отчета компилятора о распараллеливании циклов). В проекте также предусмотрен `CMakeLists.txt`, который позволяет запускать локальные тесты через стандартный `g++` без поддержки OpenACC — это использовалось для отладки логики, проверки граничных условий и корректности формата вывода.

Используемый профилировщик

В качестве инструмента профилирования выбран NVIDIA Nsight Systems. Чтобы избежать раздувания размера трассировочного файла и упростить анализ таймлайна, профильные запуски проводились с искусственным ограничением числа итераций до 50–100.

Пример команды для снятия профиля:

```
nsys profile --stats=true -o trace_gpu_512 ./heat_solver -n 512 --eps 1e-6 --max-iter 100 -o out_profile.txt
```

Как производили замер времени работы

Хронометраж выполнения вычислений встроен непосредственно в код программы с использованием стандартной библиотеки `<chrono>` (тип `high_resolution_clock`). Измеряется исключительно время итерационного процесса: засекается момент перед вызовом функции `solve(...)` и сразу после выхода из нее.

Выполнение на CPU

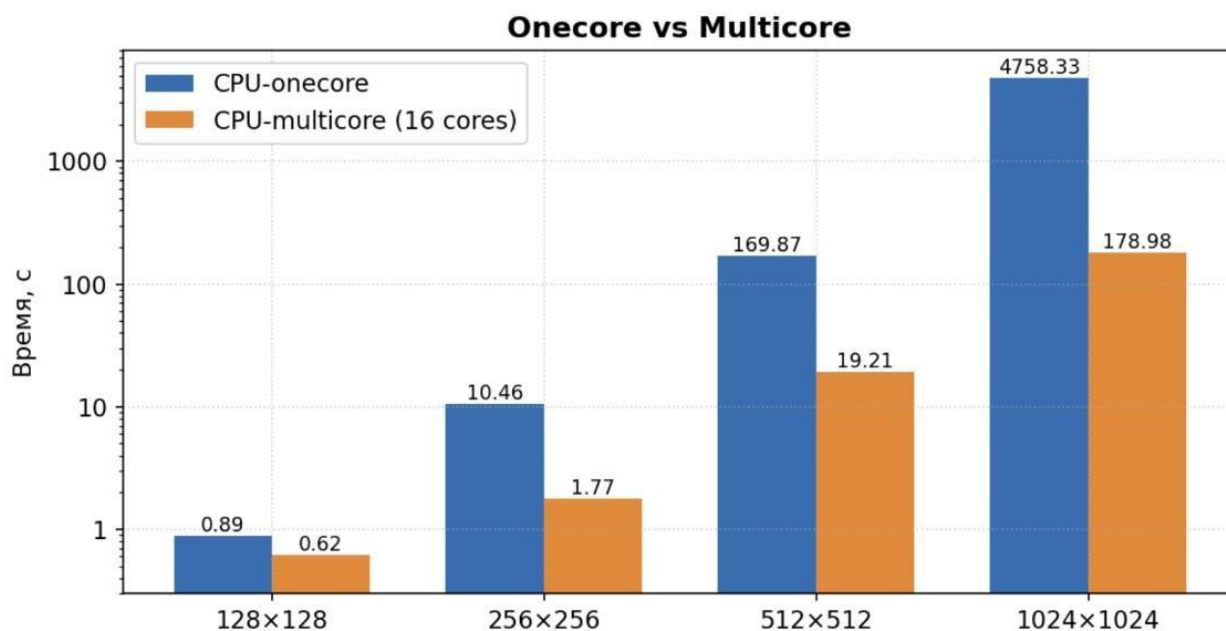
CPU-onecore

Размер сетки	Время выполнения	Точность	Количество итераций
128*128	0.89 с	9.999803e-07	30074
256*256	10.46 с	9.999302e-07	102885
512*512	169.87 с	9.999302e-07	339599
1024*1024	4758.33 с	1.36929e-06	1000000

CPU-multicore

Размер сетки	Время выполнения	Точность	Количество итераций
128*128	0.62 с	9.999302e-07	30074
256*256	1.77 с	9.999302e-07	102885
512*512	19.21 с	9.999302e-07	339599
1024*1024	178.98 с	1.36929e-06	1000000

Диаграмма сравнения время работы CPU-one и CPU-multi



Выполнение на GPU

Этапы оптимизации на сетке 512*512

Этап №	Время выполнения	Точность	Максимальное количество итераций	Комментарии (что было сделано)
1	134.2 с	9.999302e-07	1000000	Базовый вариант: использовались две матрицы current/next). На каждом шаге происходило явное копирование данных из временного массива в основной через отдельный цикл. Это создавало колоссальное узкое место при постоянном обмене данными.
2	18.7 с	9.999302e-07	1000000	Оптимизированный вариант, Применен директивный блок <code>#pragma acc data</code> , что позволило один раз скопировать сетку на устройство и забрать результат только в конце. Физическое копирование массивов заменено на swap указателей. Двумерные циклы раскрыты через <code>collapse(2)</code> , для поиска максимума невязки применен <code>reduction(max:err)</code> , а флаг <code>independent</code> снял ложные зависимости данных.

Вывод:

В ходе выполнения лабораторной работы был написан и исследован C++-алгоритм решения задачи Дирихле для уравнения теплопроводности методом простых итераций. Успешно реализованы граничные условия via линейной интерполяции, настроен парсинг аргументов через `boost::program_options` и сохранение результата в файл.

Анализ профилей и временных замеров показал, что «узким горлышком» данной задачи является не вычислительная мощность, а пропускная способность подсистемы памяти (Memory Bandwidth Bound). Отношение числа арифметических операций к количеству обращений к памяти крайне мало (на каждую итерацию требуется чтение 4-х соседних элементов и запись 1 нового).

Перенос на GPU и устранение лишних копирований между Host и Device (за счет использования data-регионов и обмена указателями вместо копирования массивов) позволил сократить время расчета более чем в 7 раз по сравнению с базовой версией на видеокарте. При этом даже оптимизированный GPU показывает ограниченную эффективность на малых сетках (128×128) из-за накладных расходов на инициализацию и передачу данных, однако на сетке 1024×1024 ускорение относительно многоядерного CPU становится весьма существенным.